

ΔΟΜΕΣ ΔΕΔΟΜΕΝΩΝ

ΑΝΑΦΟΡΑ ΠΑΡΑΔΟΣΗΣ ΕΡΓΑΣΙΑΣ 1

Στοιχεία μελών ομάδας

Ονοματεπώνυμο	Αριθμός Μητρώου
ΜΟΝΟΧΟΛΙΑΣ ΙΩΑΝΝΗΣ	3230128
ΜΠΑΝΟΣ ΒΑΣΙΛΕΙΟΣ	3230132

ΜΕΡΟΣ Α

Στο πρώτο μέρος της εργασίας πραγματοποιήθηκε η υλοποίηση των διεπαφών `StringStack` και `DoubleQueue` που δόθηκαν.

Κατ' αρχάς, κατά την υλοποίηση και των δύο διεπαφών χρησιμοποιήθηκε λίστα μονής σύνδεσης, με κόμβους αντικείμενα της κλάσης `Node<T>`, που δημιουργήθηκε από την αρχή σε δικό της αρχείο και η υλοποίησης φαίνεται παρακάτω.

```
class Node<T>{  
    private T data;  
    private Node<T> next = null;  
    public Node(T data){  
        this.data = data;  
    }  
    public void setNext(Node<T> nextNode){this.next = nextNode;}  
    public void setData(T newData){this.data = newData;}  
    public T getData(){return data;}  
    public Node<T> getNext(){return next;}  
}
```

Περιγραφή κλάσης `Node<T>`

Όπως φαίνεται η κλάση `Node<T>` κατασκευάστηκε χρησιμοποιώντας Generics, ώστε το πεδίο `data` να μπορεί να είναι δεδομένο οποιουδήποτε τύπου. Περιέχει επίσης το πεδίο `next` που δείχνει στον επόμενο κόμβο, καθώς και `public` μεθόδους πρόσβασης και ενημέρωσης αυτών των πεδίων.

• Υλοποίηση διεπαφής `StringStack`

Η διεπαφή `StringStack` χρησιμοποιήθηκε για να οριστεί η κλάση `StringStackImpl` που αναπαριστά στοίβα δεδομένων `String`. Έχει δύο πεδία: α) το `private Node<String> head` που αναπαριστά την κεφαλή της λίστας και εν προκειμένω την κορυφή της στοίβας από όπου γίνονται `push` και `pop` στοιχεία και β) Το `private int size` που δείχνει πόσα στοιχεία έχει μέσα της η στοίβα όταν εκτελείται το πρόγραμμα (η στοίβα δεν έχει σταθερό μέγεθος, εφόσον χρησιμοποιείται λίστα μονής σύνδεσης για την υλοποίησή της).

Παρακάτω περιγράφεται η λειτουργία των μεθόδων της διεπαφής `StringStack` που υλοποιήθηκαν από την `StringStackImpl`.

public void push(String item)

Κατασκευάζει έναν κόμβο `Node<String>` `tempNode` με δεδομένα `item`, θέτει την κεφαλή (στοιχείο που εισήχθη τελευταίο) να δείχνει σε αυτόν και προσauξάνει το `size` κατά 1.

Σημείωση: Αν η στοίβα είναι άδεια, απλά τίθεται η κεφαλή ίση με τον `tempNode`, ενώ σε αντίθετη περίπτωση τίθεται πρώτα ο επόμενος κόμβος του `tempNode` ίσος με την κεφαλή κι ύστερα η κεφαλή ίση με τον `tempNode`.

Public String pop() throws NoSuchElementException

Ελέγχει πρώτα αν η στοίβα είναι κενή, ώστε αν είναι να προκαλέσει εξαίρεση, όπως ζητείται και στην εκφώνηση, και ακολούθως αν η στοίβα έχει στοιχεία διατηρεί το περιεχόμενο του `head` (στοιχείο που εισήχθη τελευταίο) στην προσωρινή μεταβλητή `tempData` και θέτει το `head` ίσο με το επόμενο του, βγάζοντας από τη στοίβα το μέχρι πρότινος στοιχείο της κορυφής. Στο τέλος Μειώνει κατά 1 το μέγεθος της στοίβας και επιστρέφει τα δεδομένα της προσωρινής μεταβλητής `tempData`.

**public String peek() throws
NoSuchElementException**

Ελέγχει πρώτα αν η στοίβα είναι κενή, ώστε αν είναι να προκαλέσει εξαίρεση, όπως ζητείται και στην εκφώνηση. Αλλιώς επιστρέφει τα δεδομένα της κορυφής της στοίβας (του head της συνδεδεμένης λίστας)

public void printStack(PrintStream stream)

Αν η λίστα δεν είναι κενή αρχικοποιεί τη μεταβλητή String stack και διατρέχοντας τη λίστα από την κεφαλή ως το τέλος, την ενημερώνει με τα δεδομένα του κάθε κόμβου και στο τέλος την εκτυπώνει μέσω του stream. Αν η στοίβα είναι κενή, εκτυπώνει κατάλληλο μήνυμα.

public int size()

Απλά επιστρέφει την τιμή του πεδίου size (το οποίο ενημερώνεται **μόνο** από τις **push()** και **pop()**) και έτσι **επιστρέφει τον αριθμό των στοιχείων** της στοίβας σε χρόνο **O(1)**.

- **Υλοποίηση διεπαφής DoubleQueue**

Η διεπαφή DoubleQueue χρησιμοποιήθηκε για να οριστεί η κλάση DoubleQueueImpl που αναπαριστά ουρά δεδομένων Double. Έχει τρία πεδία: α) το private **Node<Double> head** που αναπαριστά την κεφαλή της λίστας και εν προκειμένω το εμπρός μέρος της ουράς από όπου εξάγονται τα παλαιότερα στοιχεία β) το private **Node<Double> tail** που αναπαριστά το πίσω μέρος της ουράς από όπου εισάγονται τα στοιχεία και γ) Το **private int size** που δείχνει πόσα στοιχεία έχει μέσα της η ουρά όταν εκτελείται το πρόγραμμα (η ουρά δεν έχει σταθερό μέγεθος, εφόσον χρησιμοποιείται λίστα μονής σύνδεσης για την υλοποίησή της).

Παρακάτω περιγράφεται η λειτουργία των μεθόδων της διεπαφής DoubleQueue που υλοποιήθηκαν από την DoubleQueueImpl.

public void put(double item)

Δημιουργεί έναν αντικείμενο τύπου Node<Double> και το εισάγει καταλλήλως στο τέλος της ουράς, προσαιξάνοντας στο τέλος το size.

public double get() throws NoSuchElementException

Σε περίπτωση που ουρά είναι άδεια προκαλεί εξαίρεση. Διαφορετικά, εξάγει το head (παλαιότερο στοιχείο της ουράς) και μειώνει κατά 1 το size.

**public void peek() throws
NoSuchElementException**

Σε περίπτωση που ουρά είναι άδεια προκαλεί εξαίρεση, αλλιώς επιστρέφει τα δεδομένα του παλαιότερου στοιχείου, δηλαδή του head.

public void printQueue(PrintStream stream)

Διατρέχει την λίστα με παρόμοιο τρόπο με αυτόν της printStack και εκτυπώνει τα στοιχεία, ξεκινώντας από το παλαιότερο (head) και φθάνοντας στο πιο πρόσφατο (tail). Αντίστοιχα, γίνεται εκτύπωση μηνύματος σε περίπτωση άδειας ουράς.

Public int size()

Παρόμοια με την size() της στοίβας επιστρέφει το ενημερωμένο από τις put και get πεδίο size σε χρόνο ανεξάρτητο του πλήθους των στοιχείων της ουράς O(1).

ΜΕΡΟΣ Β

Για το πρόγραμμα-πελάτη του δευτέρου μέρους χρησιμοποιήθηκε η υλοποίηση `StringStackImpl` του μέρους Α. Η κύρια λογική της χρήσης της στοίβας είναι η κάτωθι:

1. Διάβασμα του αρχείου μέχρι το τέλος του με ένα μεγάλο `while loop` που τερματίζει όταν βρεθεί `null` γραμμή στο `html` αρχείο και καθώς γίνεται η ανάγνωση:
2. Αν βρεθεί `opening tag` γίνεται `push` στην κορυφή της στοίβας(εκτός και αν είναι ένα εκ των `<img>`, `<br>` που είναι αποθηκευμένα στον πίνακα `singleTags` και γίνεται κάθε φορά έλεγχος αν το `tag` που διαβάστηκε ανήκει σε αυτόν)
3. Αν βρεθεί `closing tag` γίνεται έλεγχος αν είναι ίσο με το `top` της στοίβας
 - Αν ναι : τότε γίνεται `pop` του `tag` στην κορυφή της στοίβας
 - Αν όχι : τότε ουσιαστικά σταματάει το διάβασμα του αρχείου και εκτυπώνεται ότι οι ετικέτες είναι λάθος ταιριασμένες
4. Προς το τέλος του προγράμματος -έξω από το μεγάλο `while loop`- ελέγχεται(μέσω της `boolean` μεταβλητής `wrongMatching`) αν δεν βρέθηκε κάποιο μη σωστά ταιριασμένο `tag` και επίσης αν η στοίβα άδειασε και αν ισχύουν τα δύο προηγούμενα, τότε τυπώνεται ότι το `html` αρχείο έχει σωστά ταιριασμένες ετικέτες και το αντίστοιχο μήνυμα λάθος ταιριάσματος των ετικετών αν δεν ισχύει κάποια από τις παραπάνω συνθήκες.

Παρατηρούμε, λοιπόν, ότι για την υλοποίηση του προγράμματος είναι απαραίτητος ο τρόπος οργάνωσης των δεδομένων **LIFO** που υποστηρίζει η στοίβα, ώστε να μπορεί να ελέγχεται ότι το `tag` που άνοιξε τελευταίο κλείνει πρώτο, σε περίπτωση που έχουμε `nested tags`. Διαφορετικά, αν δεν ισχύει το παραπάνω στο αρχείο `html` έστω και για ένα ζεύγος `opening` και `closing nested tags`, τυπώνεται ότι τα `tags` σε όλο το αρχείο δεν είναι σωστά ταιριασμένα.

Στην περίπτωση, τώρα, που δεν εξετάζονται `nested tags`(είτε γιατί δεν υπάρχουν στο αρχείο είτε γιατί υπάρχουν και είναι σωστά ταιριασμένα), αλλά έχουμε `μη nested tags`, γίνεται έλεγχος αν το συγκεκριμένο `tag` μόνο ανοίγει και δεν κλείνει ποτέ ή δεν ανοίγει ποτέ αλλά εμφανίζεται ένα `closing tag`, ώστε να τυπώνεται μήνυμα λάθους.

Κάποιες λεπτομέρειες για τον κώδικα:

- Το όνομα του `html` αρχείου δίνεται ως όρισμα στο `command line` ως `java TagChecking htmlFile.html`
- Για την ανάγνωση του αρχείου χρησιμοποιείται η κλάση της `Java` **`BufferedReader`**
- Χρησιμοποιήθηκαν αρκετές μέθοδοι της `java` για επεξεργασία και έλεγχο σε `string` (`trim()`, `substring()`, `length()`, `equals()`, `contains()`, `charAt()`) με τα αντίστοιχα ορίσματά τους προφανώς.
- Το πρόγραμμα συλλαμβάνει τις εξαιρέσεις, αν αυτές εμφανιστούν.
- Υπάρχουν αναλυτικά σχόλια σε όλη την έκταση του κώδικα με λεπτομέρειες για τη χρήση συγκεκριμένων μεταβλητών, συνθηκών, βρόχων

ΜΕΡΟΣ Γ

Για το πρόγραμμα μετοχών του τρίτου μέρους χρησιμοποιήθηκε η υλοποίηση `DoubleQueueImpl` του μέρους Α. Η κύρια λογική της χρήσης της ουράς είναι η κάτωθι:

1. Διάβασμα του αρχείου μέχρι το τέλος του με ένα μεγάλο `while loop` που τερματίζει όταν βρεθεί `null` γραμμή στο `txt` αρχείο και καθώς γίνεται η ανάγνωση:
2. Αν βρεθεί κενή γραμμή στο αρχείο, ενώ υπάρχουν και άλλες γραμμές για ανάγνωση, παραλείπεται και πάει στην επόμενη γραμμή.
3. Στην συνέχεια, ανάλογα με την πρώτη λέξη της γραμμής που διαβάζει, καταλαβαίνουμε αν θα αγοράσει μετοχές ή θα πουλήσει, ώστε κατά σειρά να χρησιμοποιήσουμε με κατάλληλο τρόπο την ουρά.
 - Αν είναι “buy”, με τη βοήθεια επόμενων ελέγχων και του αντικειμένου `tokenizer` εκχωρούμε στο τέλος της ουράς τον κατάλληλο αριθμό μετοχών με την ανάλογη τιμή από την οποία τα αγόρασε. Σε αυτό μας βοηθάει και η δομή επανάληψης `for`.
 - Αν είναι “sell”, με τη βοήθεια επόμενων ελέγχων και του αντικειμένου `tokenizer` εξαγούμε από την αρχή της ουράς το κατάλληλο αριθμό μετοχών (εφόσον αυτές είναι διαθέσιμες, ενώ σε διαφορετική περίπτωση το πρόγραμμα εμφανίζει κατάλληλο μήνυμα λάθους και συνεχίζει το διάβασμα στην επόμενη γραμμή) με την ανάλογη τιμή από την οποία τα αγόρασε με σκοπό να υπολογίσουμε το κέρδος ή τη ζημία. Σε αυτό μας βοηθάει και η δομή επανάληψης `for`.
4. Τέλος, εφόσον το αρχείο δεν έχει άλλες γραμμές για διάβασμα, με τη βοήθεια της μεταβλητής `netProfit`, εμφανίζουμε το κέρδος ή την ζημία του πελάτη, έπειτα από τις αγοραπωλησίες των μετοχών.

Παρατηρούμε, λοιπόν, ότι για την υλοποίηση του προγράμματος είναι απαραίτητος ο τρόπος οργάνωσης των δεδομένων **FIFO** που υποστηρίζει η ουρά, ώστε να μπορεί να ελέγχεται ότι η μετοχή που θα αγοραστεί πρώτη θα πουληθεί και πρώτη, και με τη σειρά και οι επόμενες μετοχές.

Κάποιες λεπτομέρειες για τον κώδικα:

- Το όνομα του `txt` αρχείου δίνεται ως όρισμα στο `command line` ως εξής: `java NetProfit trades.txt`
- Για την ανάγνωση του αρχείου χρησιμοποιείται η κλάση της Java **`BufferedReader`**
- Χρησιμοποιήθηκαν αρκετές μέθοδοι της Java για επεξεργασία και έλεγχο σε `string` (`trim()`, `length()`, `equals()`...) με τα αντίστοιχα ορίσματά τους προφανώς.
- Το πρόγραμμα συλλαμβάνει τις εξαιρέσεις, αν αυτές εμφανιστούν.
- Υπάρχουν αναλυτικά σχόλια σε όλη την έκταση του κώδικα με λεπτομέρειες για τη χρήση συγκεκριμένων μεταβλητών, συνθηκών, βρόχων